

Dynamic Programming (Part III)

Weighted Interval Scheduling

(Kleinberg and Tardos, 6.1)

Weighted Interval Scheduling

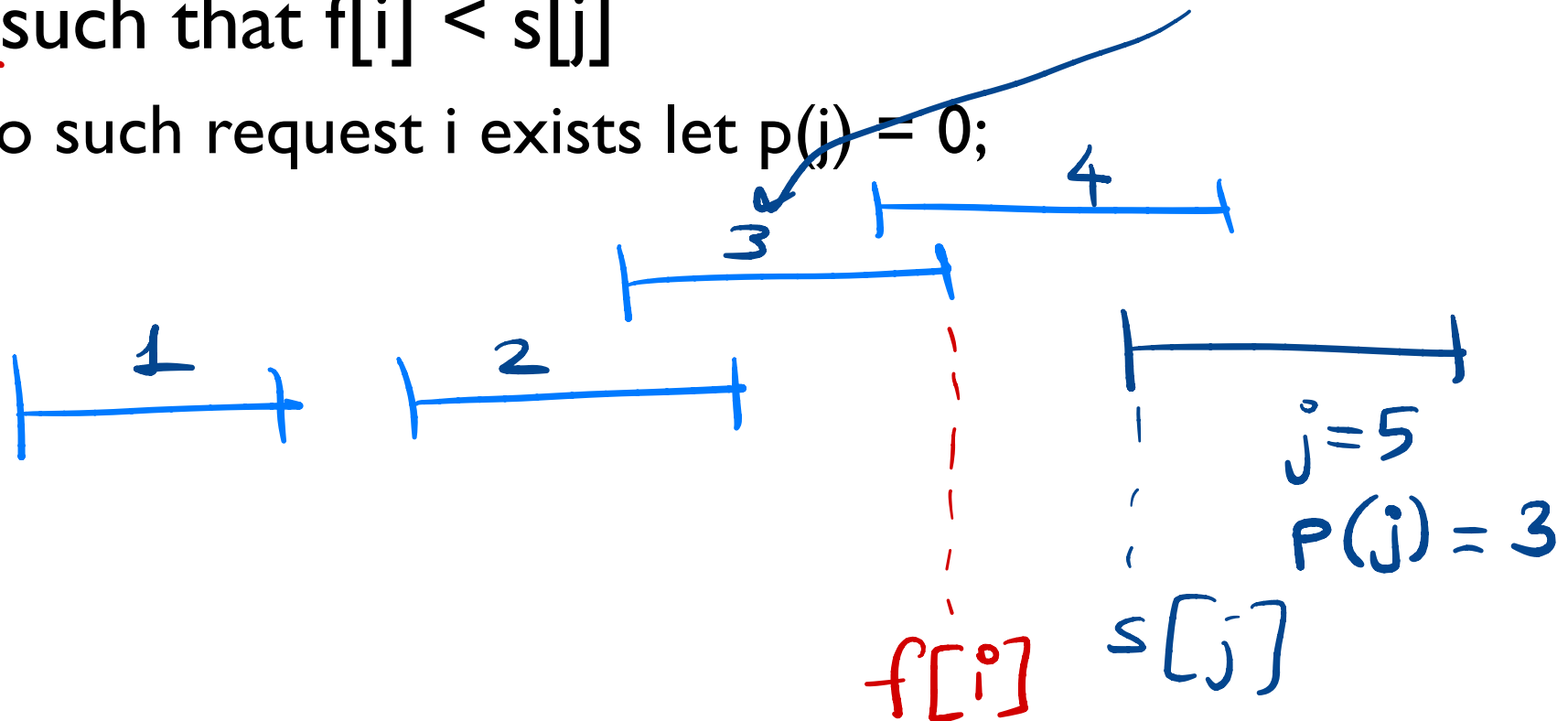
- Recall the original interval scheduling problem:
 - Given $R=\{1,2,\dots,n\}$ of n activity requests to be scheduled to use a single resource
 - i -th request should start at time $s[i]$ and end at time $f[i]$
 - Select a maximum size set of mutually non-interfering activities

Weighted Interval Scheduling

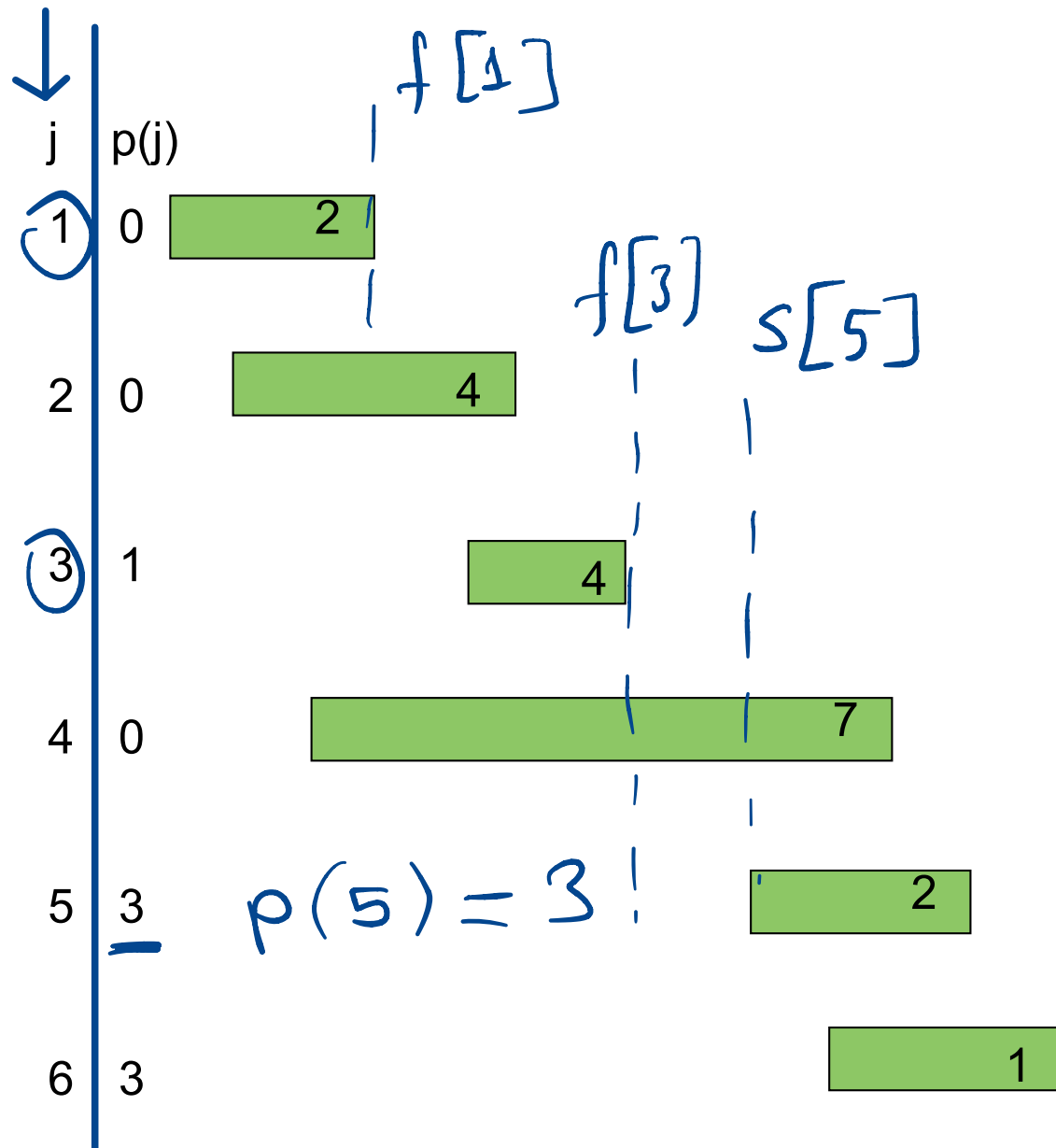
- Weighted Interval Scheduling (WIS)
 - In addition, each request is associated with a numeric value $v[i]$
 - Find a set of non-interfering request such that the sum of the values is maximum

No greedy solution—try DP

- First sort requests in increasing finish time
- Given a request j , define $p(j)$ to be the largest $i < j$ such that $f[i] < s[j]$
 - If no such request i exists let $p(j) = 0$;



$p(j)$: the largest $i < j$ such that $f[i] < s[j]$



DP Formulation?

- Let's first compute the optimum total value and then extract the set of requests that produce that value

parameter $\rightarrow$ no need to use a parameter for the starting index, since it is always 1.

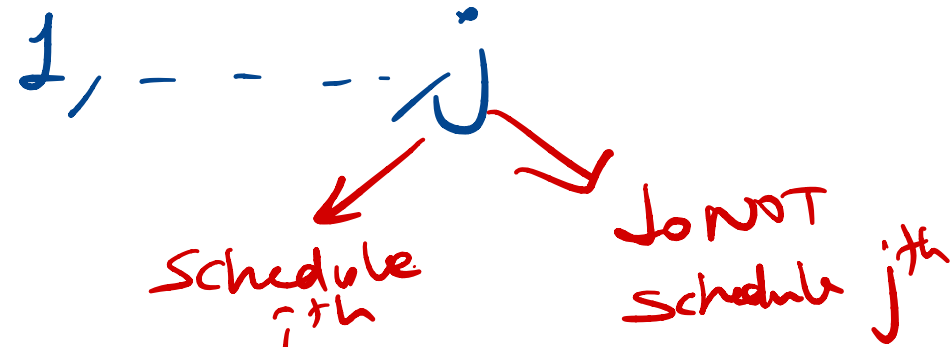
- $M[j]$: maximum possible value if we restrict attention to first j requests only

- Global problem: $M[n]$

DP formulation:

- Basis: $M[0] = 0$

DP formulation:

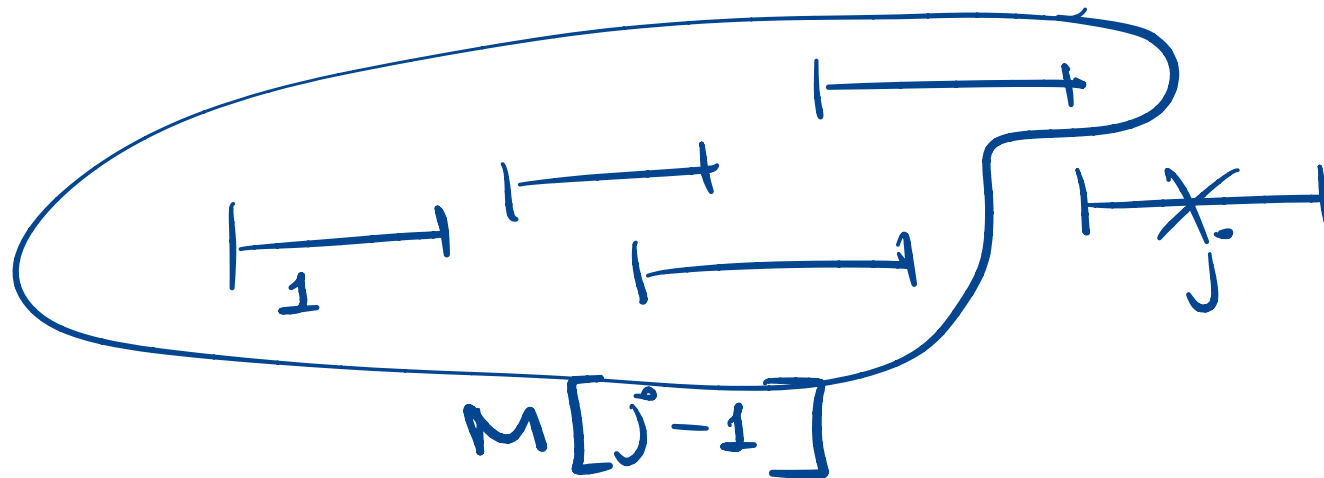


- $M[j]$, for an arbitrary j , $1 \leq j \leq n$

Two cases:

CASE 1: Request j is not in the schedule: do the best you can with the first $j-1$ requests

$$M[j] = M[j-1]$$

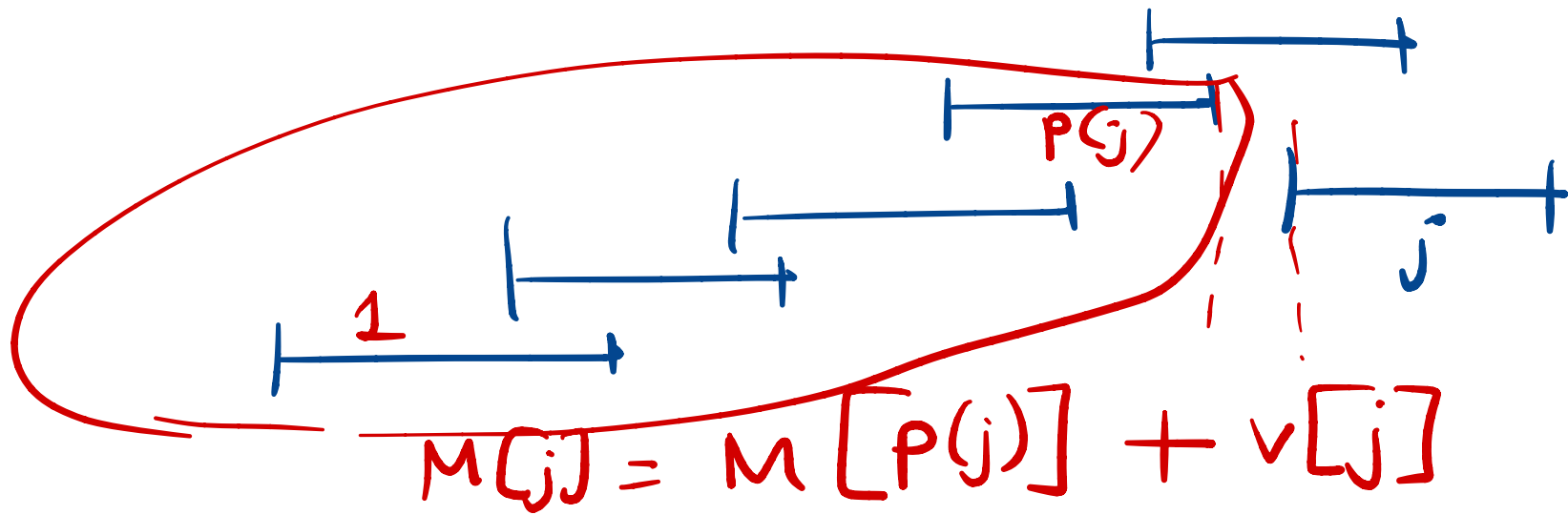


DP formulation:

CASE 2: Request j is in the schedule

- Add $v[j]$ to the total value
- We cannot take any requests following $p(j)$

$$M[j] = v[j] + M[p(j)]$$



DP Formulation

- Pick the best of CASE 1 and CASE 2

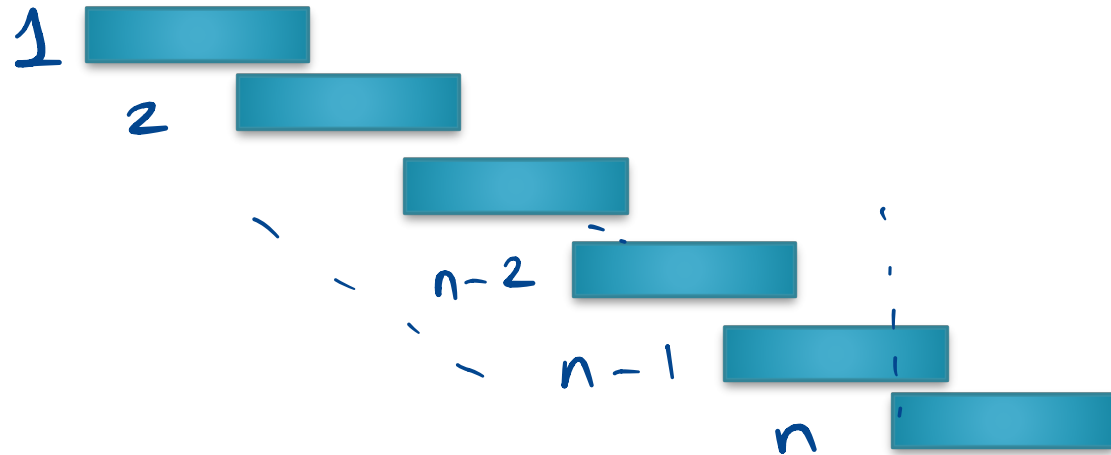
$$M[j] = \max \{M[j-1], M[p(j)] + v[j]\}$$

Optimal Substructure Property?

- Yes, it holds.
- To compute $M[j]$ optimally, the two subproblems, $M[j-1]$ and $M[p(j)]$ should be computed optimally.

Exercise

- Analyze the following recursive procedure in the worst case: e.g. consider intervals as in:



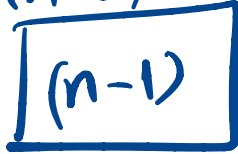
```
recursiveWIS(j) {  
    if (j == 0) return 0;  
    else return max(recursiveWIS(j-1), v[j]+recursiveWIS(p[j]) );  
}
```

recursive WLS (n)

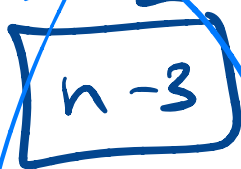
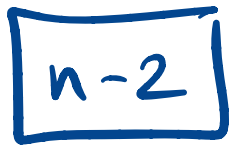
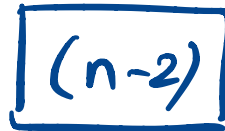
$T(n)$ input size



$T(n-1)$



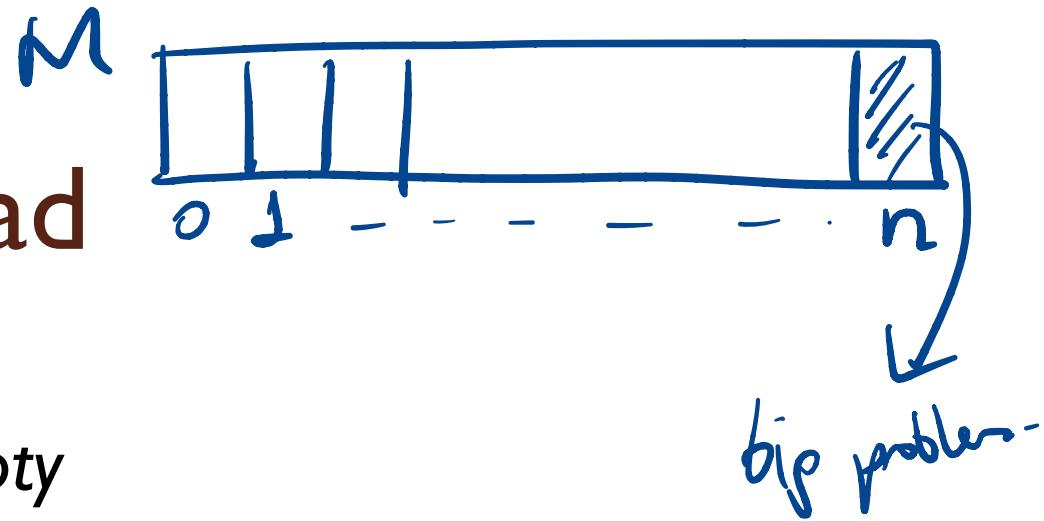
$T(n-2)$



overlapping subproblems

$T(n)$ is exponential.

Memoization instead



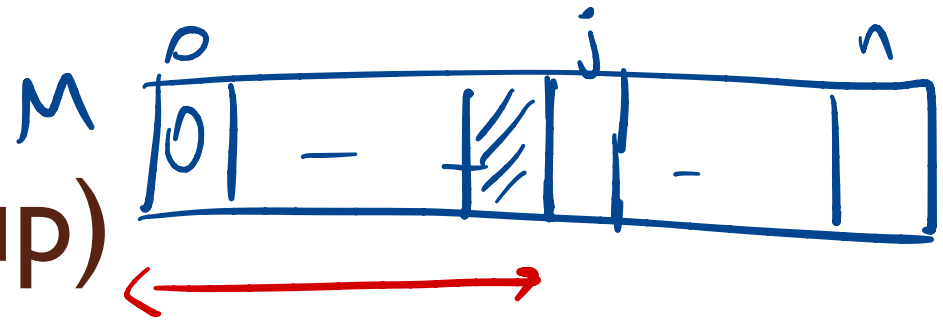
- Recursive (top-down)
- $M[1..n]$: initialize all to *empty*

```
memoized-recursiveWIS(j) {  
    if (j == 0) return 0;  
    else if ( M[j] is not empty ) return M[j] //already computed  
    else  
        ↓ return (M[j]=max(memoized-recursiveWIS(j-1),  
                           v[j]+memoizedrecursiveWIS(p[j])));  
}
```

Memoization instead

- Assume the intervals are sorted by their finish times
- Time spent in a single memoized-recursiveWIS is $O(I)$
 - either returns a computed entry in $O(1)$ time
 - or computes one new entry by making recursive calls.
- Each empty $M[i]$ causes two recursive calls, and **decreases** the number of empty $M[i]$'s by 1.
- M has $n+1$ empty entries total initially, and after $n+1$ decrements there will be 0 empty entries
- So there can be at most $O(n)$ recursive calls

Iterative (bottom-up)



$p(j)$
could be anywhere in
this region.

```
memoized-iterativeWIS() {
```

```
    M[0] = 0;
```

```
    for j = 1 to n do
```

```
        if ( $M[j-1] > v[j] + M[p[j]]$ )
```

```
            M[j] = M[j-1];
```

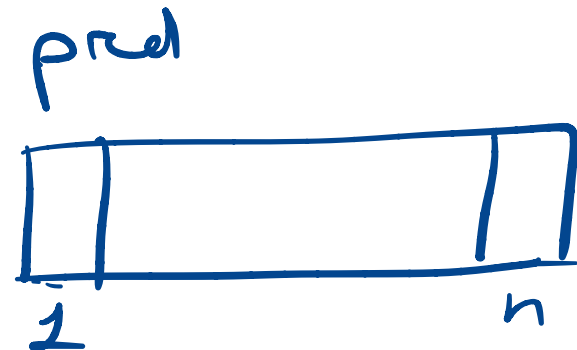
```
            pred[j] = j-1; // to remember our choice
```

```
        else
```

```
            M[j] = v[j] + M[p[j]];
```

```
            pred[j] = p[j];
```

```
}
```



Example

<u>j</u>	<u>p(j)</u>	
1	0	2
2	0	4
3	1	4
4	0	7
5	3	2
6	3	1

input

	1	2	3	4	5	6
v	2	4	4	7	2	1

	0	1	2	3	4	5	6
M	0	2					

	1	2	3	4	5	6
pred	0					

$$M[i] = \max(M[0], \underbrace{v[1]}_2 + \underbrace{M[p(1)]}_0)$$

$$M[1] = 2$$

Example

<u>j</u>	<u>p(j)</u>	
1	0	2
2	0	4
3	1	4
4	0	7
5	3	2
6	3	1

	1	2	3	4	5	6
v	2	4	4	7	2	1

	0	1	2	3	4	5	6
M	0	2	4				

$p(1)$ points to index 1 in the 'pred' array.
 $p(2)$ points to index 2 in the 'pred' array.

	1	2	3	4	5	6
pred	0	0				

$$M[2] = \max(M[1], v[2] + M[p(2)])$$

Handwritten calculation for $M[2]$:
 $M[2] = \max(2, 4 + 0) = 4$

Final result: $M[2] = 4$

Example

j p(j)

1 0 2

2 0 4

3 1 4

4 0 7

5 3 2

6 3 1

request 3 is scheduled ←

request 3 not scheduled ←

	1	2	3	4	5	6
v	2	4	4	7	2	1

	0	1	2	3	4	5	6
M	0	2	4	6			

	1	2	3	4	5	6
pred	0	0	1			

$$M[3] = \max(M[2], v[3] + M[p(3)])$$

$$M[3] = 6$$

Handwritten annotations:
 - Blue arrow from $p(3)$ to index 1 in pred array.
 - Red arrow from $M[2]$ to 4.
 - Red arrow from $v[3]$ to 4.
 - Red arrow from $M[p(3)]$ to $M[1] = 2$.
 - Blue circle around $v[3] + M[p(3)]$.
 - Red underline under $M[2]$.
 - Red underline under $M[p(3)]$.

Example

j p(j)

1 0 2

2 0 4

3 1 4

4 0 7

5 3 2

6 3 1

	1	2	3	4	5	6
v	2	4	4	7	2	1



	0	1	2	3	4	5	6
M	0	2	4	6	7		

	1	2	3	4	5	6
pred	0	0	1	0		

$$M[4] = (M[3] + v[4] + M[p(4)])$$

$$M[4] = 7 + 7 + 0$$

Handwritten calculation showing the update of M[4] using M[3], v[4], and p(4). A blue arrow points from p(4) to the value 0 in the pred array at index 4. A red arrow points from the value 0 in the pred array at index 4 to the M[p(4)] term in the formula. The final result M[4] = 7 is shown with a red underline.

Example

j p(j)

1 0 2

2 0 4

3 1 4

4 0 7

5 3 2

6 3 1

	1	2	3	4	5	6
v	2	4	4	7	2	1

	0	1	2	3	4	5	6
M	0	2	4	6	7	8	

	1	2	3	4	5	6
pred	0	0	1	0	3	

Example

j p(j)

1 0 2 ✓

2 0 4 ✗

3 1 ✓ 4

4 0 7 ✗

5 3 ✓ 2

6 3 ✗ 1

	1	2	3	4	5	6
v	2	4	4	7	2	1

	0	1	2	3	4	5	6
M	0	2	4	6	7	8	8

	1	2	3	4	5	6
pred	0	0	1	0	3	5

$$M[6] = \max \left(\underbrace{M[5]}_8, \underbrace{v[6] + M[3]}_{1+6} \right) = 8$$

How do we extract which intervals are scheduled?

- 8 is the value of the optimal solution
- Which set of intervals gave us the value 8?
- Can be done using pred or without using it just by looking at values of M

Exercise : extraction algorithm.